LiteTest Test Reference

This document summarizes the public Test API declared in `include/litetest_test.h`. It focuses on the helper-specific return conventions and the option structs used by the comparison helpers.

Preface

This document is intended for LiteTest users and contributors who need quick access to the definitions of terms used in LiteTest or to browse through the terms.

For a list of other LiteTest documents and the LiteTest repository layout, see the LiteTest Documentation Guide.

For a glossary of terms, see the LiteTest Glossary Reference.

Document Version History

Document	Date	Runner ! Test	Comment	Author/Editor
1.0	2026-06-11	1.0.0	1.0.0 ! Initial version.	Paul Sinclair

The `Document` column uses the version format `M.u` (Major, update). The `Runner` column records the LiteTest Runner version current at the time this document version was published. The `Test` column records the LiteTest Test version current at the time this document version was published.

The current LiteTest Runner version is defined by the `LT_RUNNER_VERSION` macro in the LiteTest RUNNER API. The current LiteTest Test version is defined by the `LT_TEST_VERSION` macro in the LiteTest Test API. Both specify a string of the form `"M.m.p"` (Major, minor, patch). The

The document's `M` (Major) version matches the LiteTest Runner's `M` (Major) version which matches the LiteTest Test's `M` (Major) version. The document's `u` (update) version track updates to this document and does not correspond to a `m` (minor) or `p` (patch) version. `u` increments whenever this document is updated without a change to `M`, and it resets to `0` when `M` is incremented.

1. Return Conventions

1.1 Return Styles

LiteTest test helpers use three return styles:

- Structured helpers return `LT_OK`, `LT_MISMATCH`, `LT_TIMEOUT`, or an `LT_E*` error code.
- Predicate helpers return `1` for true or match, `0` for false or no match, and `-1` on invalid input or runtime error.
- Process helpers that model command completion return `0` for success, `1` for timeout, and `-1` for invalid input or setup failure.

1.2 Assertion Guidance

Use the helper family to decide what to check:

- File, JSON, metadata, and normalized-text comparisons should usually be checked against `LT_OK` or `LT_MISMATCH`.
- Existence, string, wildcard, and regex predicates should usually be checked against `1` or `0`.
- Command-execution helpers should usually be checked against `0` or `1`.

2. Comparison Option Structs

The comparison helpers use small option structs instead of mixed boolean and flag parameter lists.

2.1 lt_path_compare_options_t

Used by `lt_compare_paths_with_options`.

```
typedef struct {
    int flags;
} lt_path_compare_options_t;
```

Supported flags:

- `LT_FILECMP_IGNORE_TRAILING_WHITESPACE`
- `LT_FILECMP_IGNORE_EMPTY_LINES`
- `LT_FILECMP_IGNORE_TIMESTAMPS`
- `LT_FILECMP_IGNORE_LINE_ENDINGS`
- `LT_FILECMP_CASE_INSENSITIVE`

Default initializer:

```
#define LT_PATH_COMPARE_OPTIONS_INIT {0}
```

2.2 lt_path_metadata_compare_options_t

Used by `lt_compare_path_metadata`.

```
typedef struct {
    int flags;
} lt_path_metadata_compare_options_t;
```

Supported flags:

- `LT_STATCMP_SIZE`
- `LT_STATCMP_PERMS`
- `LT_STATCMP_MTIME`
- `LT_STATCMP_OWNER`
- `LT_STATCMP_TYPE`

Default initializer:

```
#define LT_PATH_METADATA_COMPARE_OPTIONS_INIT {0}
```

Passing `NULL` or the default initializer uses the common default metadata set: size, permissions, and type.

2.3 lt_json_compare_options_t

Used by `lt_compare_json_with_limit`.

```
typedef struct {
    size_t max_bytes;
```

```
    int ignore_key_order;
} lt_json_compare_options_t;
```

Default initializer:

```
#define LT_JSON_COMPARE_OPTIONS_INIT {0, 0}
```

Use `ignore_key_order` when semantic JSON equality matters more than source ordering. Use `max_bytes` when the caller wants a tighter bound than the implementation default.

2.4 `lt_text_compare_options_t`

Used by `lt_compare_text_normalized`.

```
typedef struct {
    int ignore_whitespace;
    int ignore_line_endings;
} lt_text_compare_options_t;
```

Default initializer:

```
#define LT_TEXT_COMPARE_OPTIONS_INIT {0, 0}
```

3. Common Usage Patterns

3.1 Compare Two Paths While Ignoring Timestamps

```
const lt_path_compare_options_t options = {
    LT_FILECMP_IGNORE_TIMESTAMPS
};

LT_ASSERT(lt_compare_paths_with_options(expected_path, actual_path, &options) == LT_OK,
```

3.2 Compare Path Metadata Including Modification Time

```
const lt_path_metadata_compare_options_t options = {
    LT_STATCMP_SIZE | LT_STATCMP_PERMS | LT_STATCMP_TYPE | LT_STATCMP_MTIME
};

LT_ASSERT(lt_compare_path_metadata(left_path, right_path, &options) == LT_OK, 0);
```

3.3 Compare JSON While Ignoring Object Key Order

```
const lt_json_compare_options_t options = {
    0,
    1
};

LT_ASSERT(lt_compare_json_with_limit(expected_json, actual_json, &options) == LT_OK, 0);
```

3.4 Compare Text While Ignoring Whitespace and Line Endings

```
const lt_text_compare_options_t options = {
    1,
    1
};

LT_ASSERT(lt_compare_text_normalized(left_text, right_text, &options) == LT_OK, 0);
```

4. Convenience Forms

- `lt_compare_paths` performs a byte-for-byte path comparison.
- `lt_compare_json` performs a strict JSON comparison using default limits.
- `lt_compare_file_lines` compares files line-by-line and returns `LT_OK` or `LT_MISMATCH`.

For the full declaration list and per-function comments, see `include/litetest_test.h`.